AWS Level 3 iteration 2 terminal documentation

Claudio Verdi | APL | 23 to 24 October 2026

I created an Ubuntu instance with Terraform and used user-data to update packages and automatically install and start Nginx. I then deployed the website manually over SSH. HTTP tests and a content test succeeded. After testing, I removed the instance and verified its terminated state.

Environment and resources

Property	Value
Region	eu-north-1
Terraform / AWS provider	1.16.3 / hashicorp/aws 6.70.0
Instance ID	i-08bb6ecd7cad083f4
Instance type	t3.micro
AMI	ami-06fb1802e2b757f09
OS	Ubuntu 24.04.5 LTS x86_64
Subnet	subnet-05d4dca4820d2eb94
Security group	sg-0fb87d0f08a213466
Key pair	devops.school.level3.tabish-admin
Former public IP	13.53.35.150
Private IP	172.31.19.103
Nginx / Git	1.24.0 / 2.43.0
Root disk	8 GiB gp3 / encrypted / delete_on_termination

Reference: Level3 Iteration2 (1)(2)(1).sh. I adapted the guide’s Amazon Linux commands to Ubuntu and the actual file structure.

Preparation and user data

I created ~/aws-devops-level3-iteration2 and copied five configuration files from iteration 1. I did not copy Terraform state. I reused the region, Ubuntu AMI, subnet, security group and key pair from the previous iteration.

user-data.web.sh uses apt-get and systemctl. cloud-init runs the script as root at first boot. bash -n checked its syntax without running the installation locally.

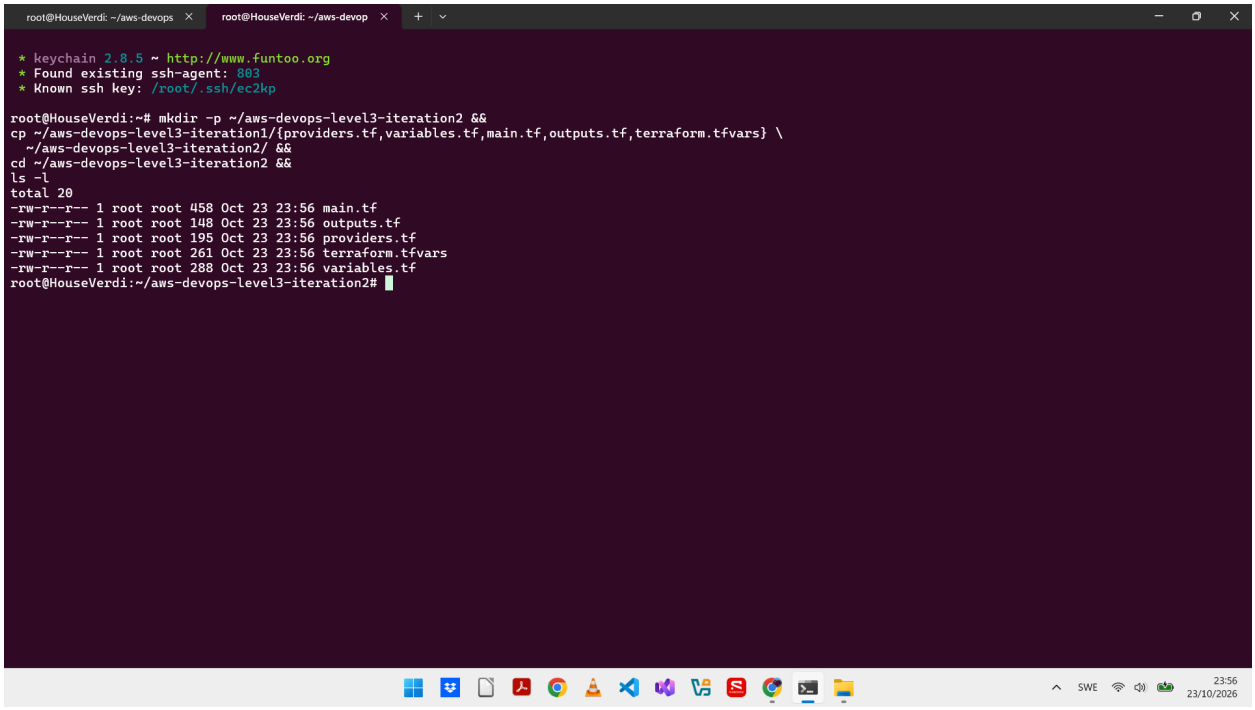


Figure 1. Five configuration files in a separate directory

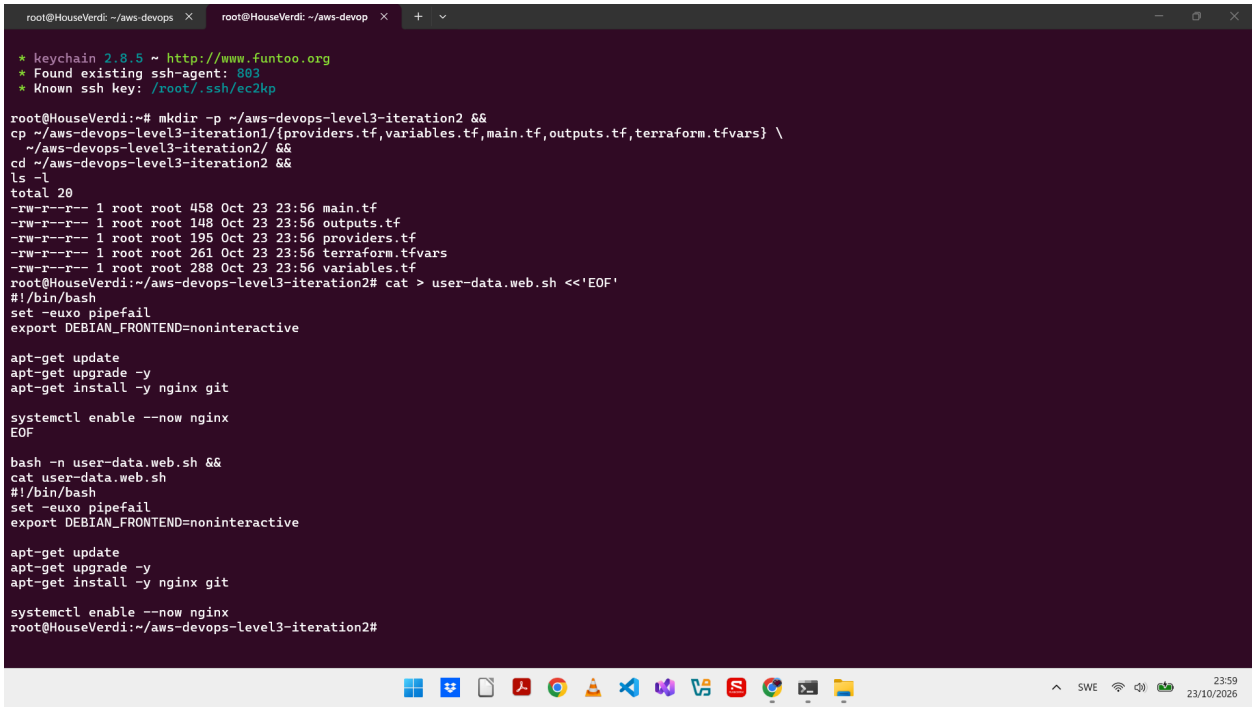


Figure 2. User-data script created and syntax checked

Terraform configuration and validation

I connected user_data to the script file and set user_data_replace_on_change=true. I retained the IMDSv2 requirement, public IP and encrypted root disk. Terraform managed only the EC2 instance; the network and key pair already existed.

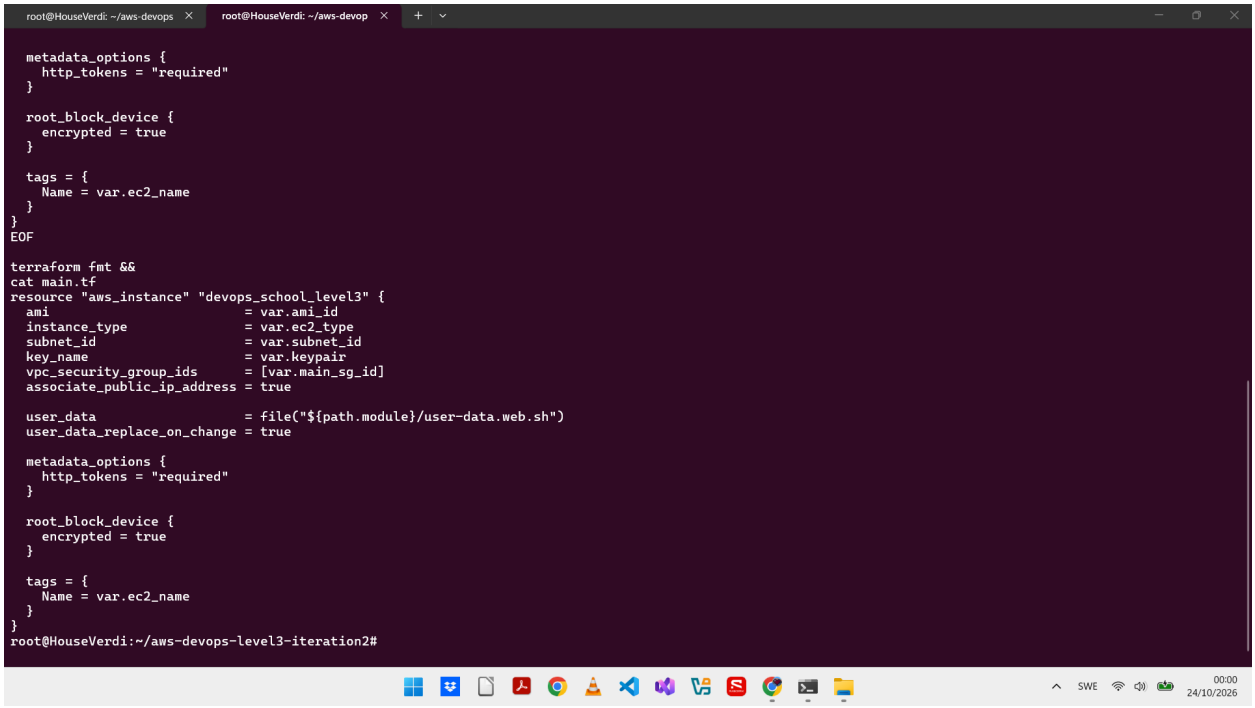


Figure 3. main.tf with user_data and replacement on changes

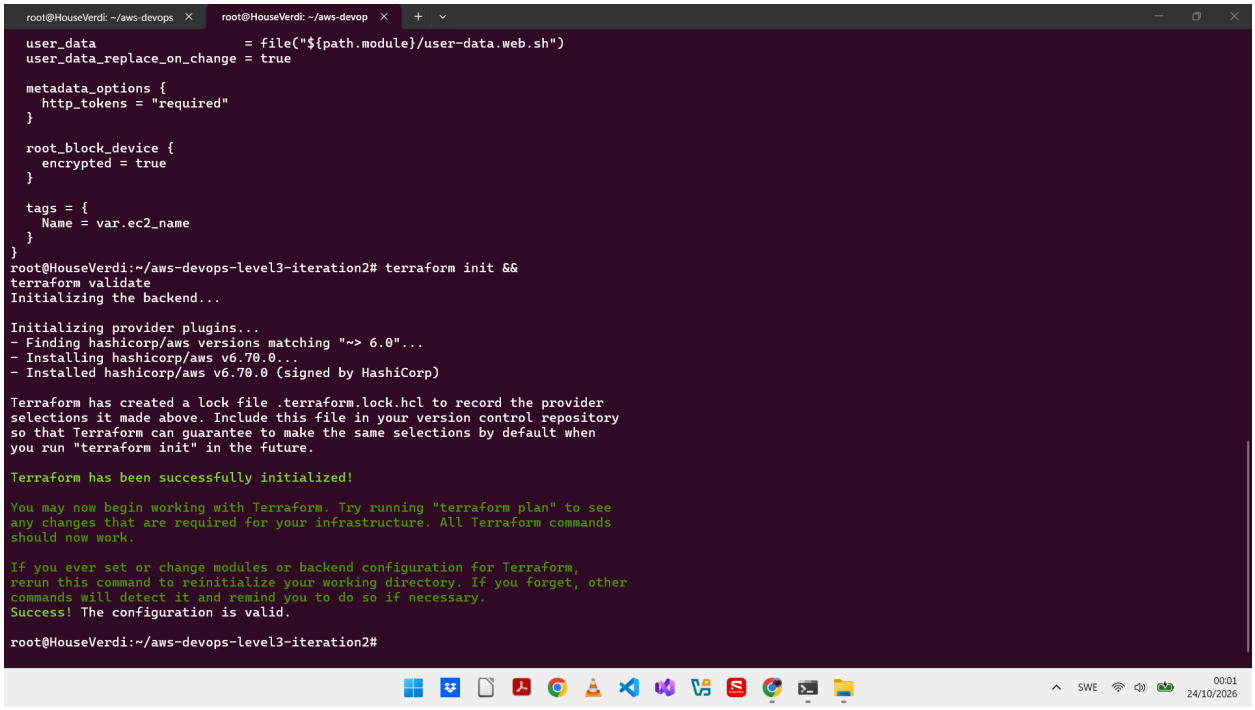


Figure 4. Initialization and successful validation with AWS provider 6.70.0

Planning and provisioning

terraform plan -out=tfplan showed 1 to add, 0 to change and 0 to destroy. terraform apply tfplan created the instance in 14 seconds and returned a public DNS name. This confirmed creation of the EC2 resource; I checked cloud-init separately.

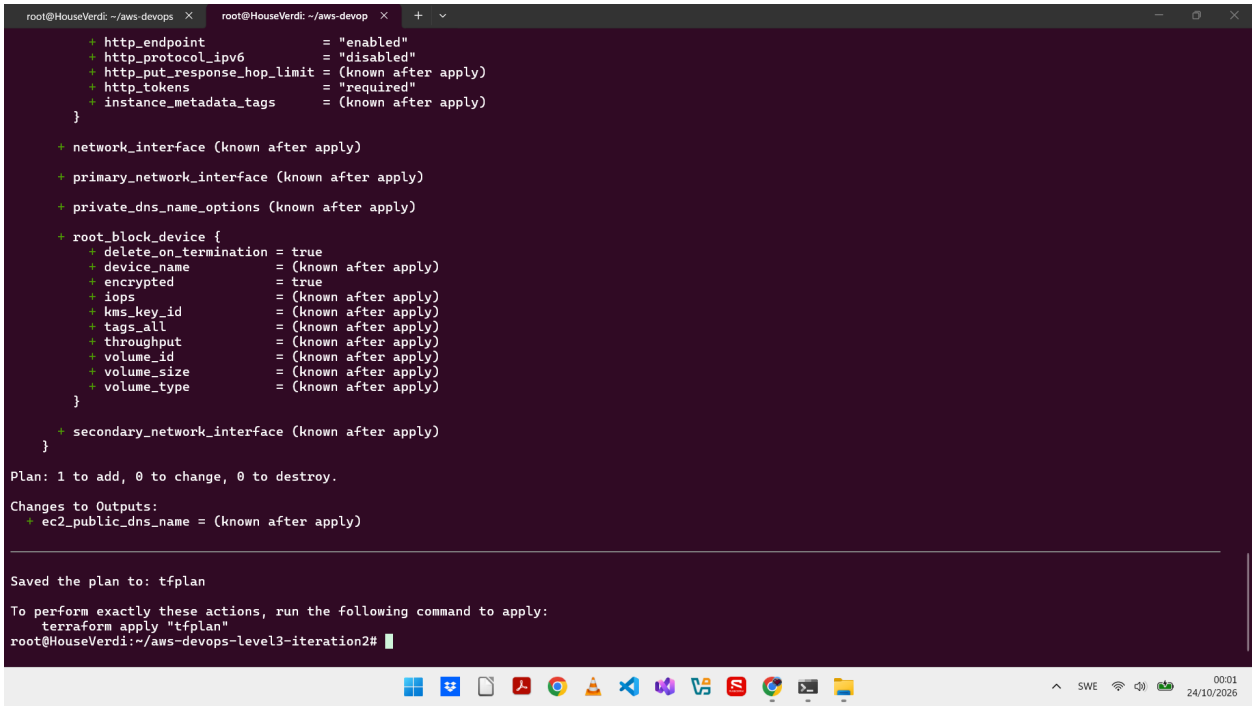


Figure 5. Saved plan with one new resource

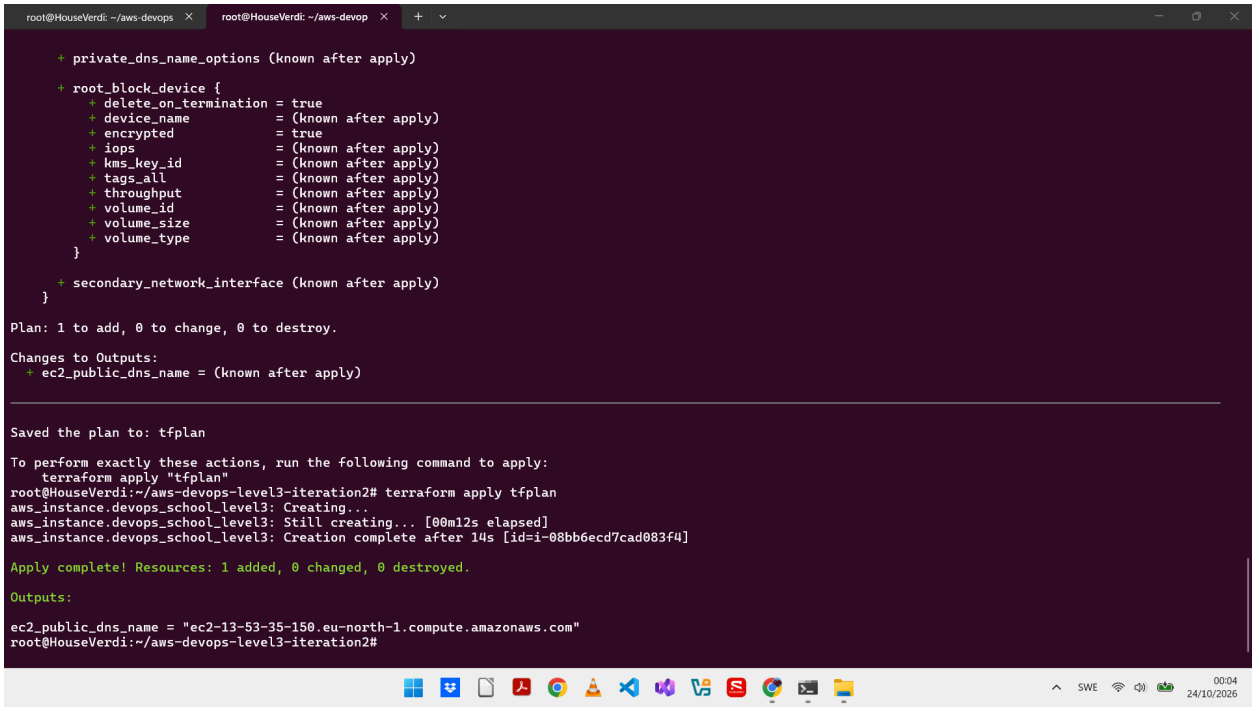


Figure 6. Apply complete with instance ID and DNS name

Verifying automatic installation

Over SSH, I ran sudo cloud-init status --wait, which returned status done. Version and service checks showed Nginx 1.24.0, Git 2.43.0, active and enabled. No manual installation commands were needed in the SSH session.

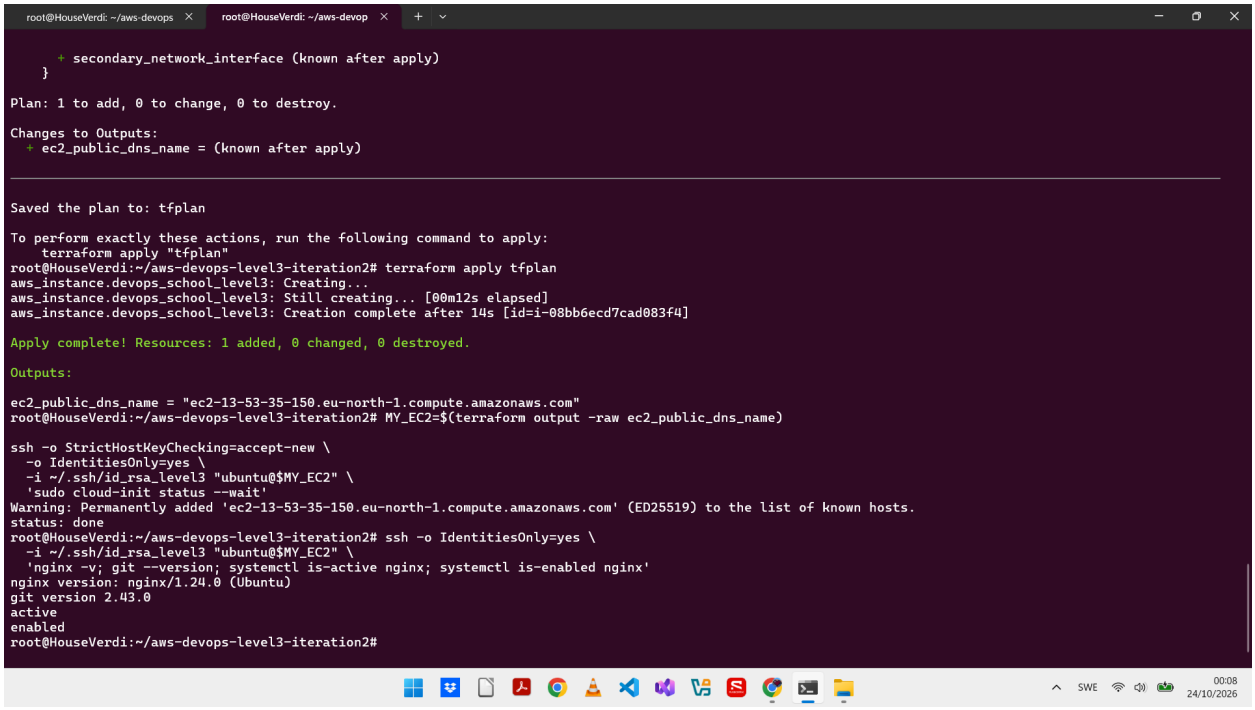


Figure 7. Completed cloud-init with verified versions and service state

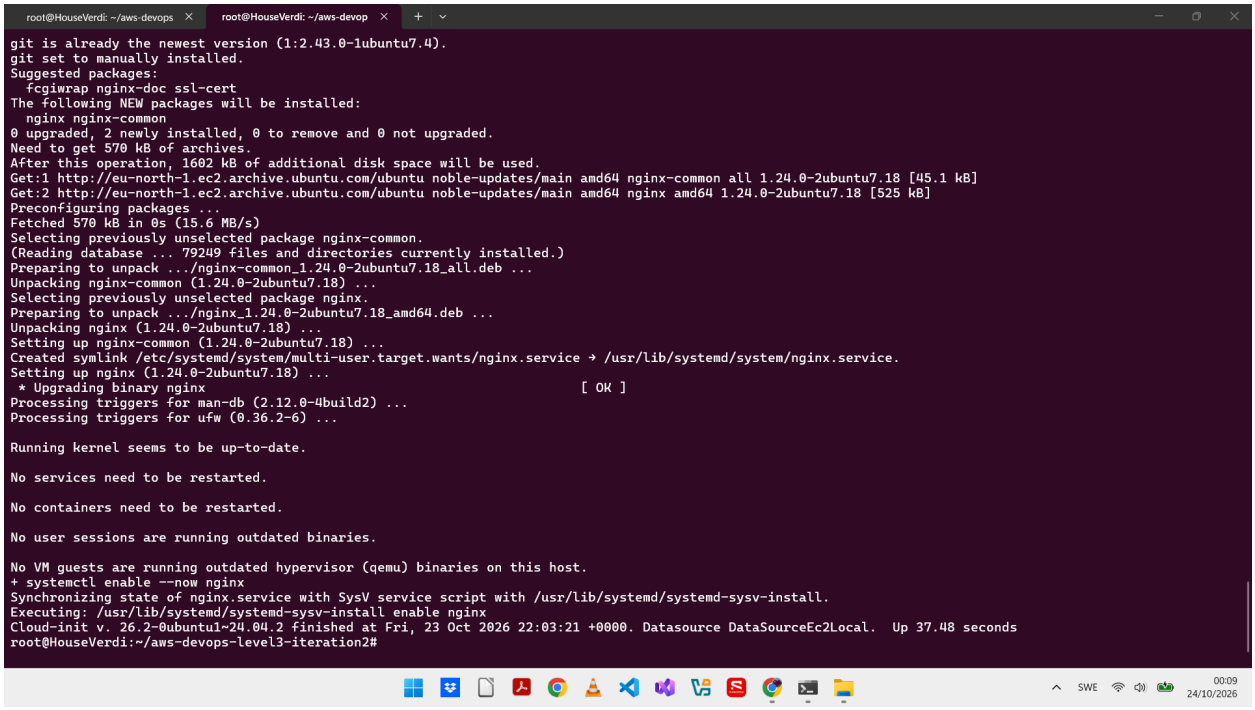


Figure 8. Installation log shows Nginx and completed cloud-init

Manual deployment over SSH

I connected interactively as ubuntu to the server with private IP 172.31.19.103 and cloned the website into ~/hello-web. The directory contained the 499-byte file heyme, but no html directory. I therefore copied heyme to /var/www/html/index.html.

sudo nginx -t returned syntax is ok and test is successful. I then reloaded Nginx.

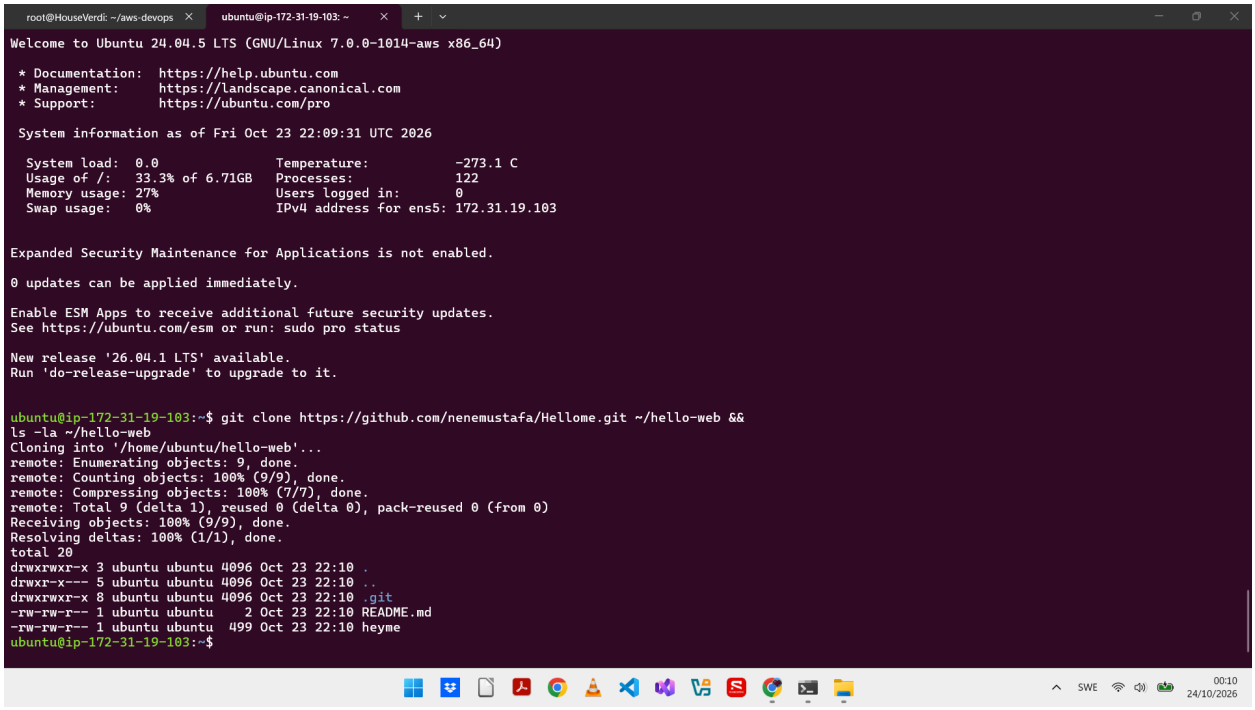


Figure 9. Clone and inspection of website files

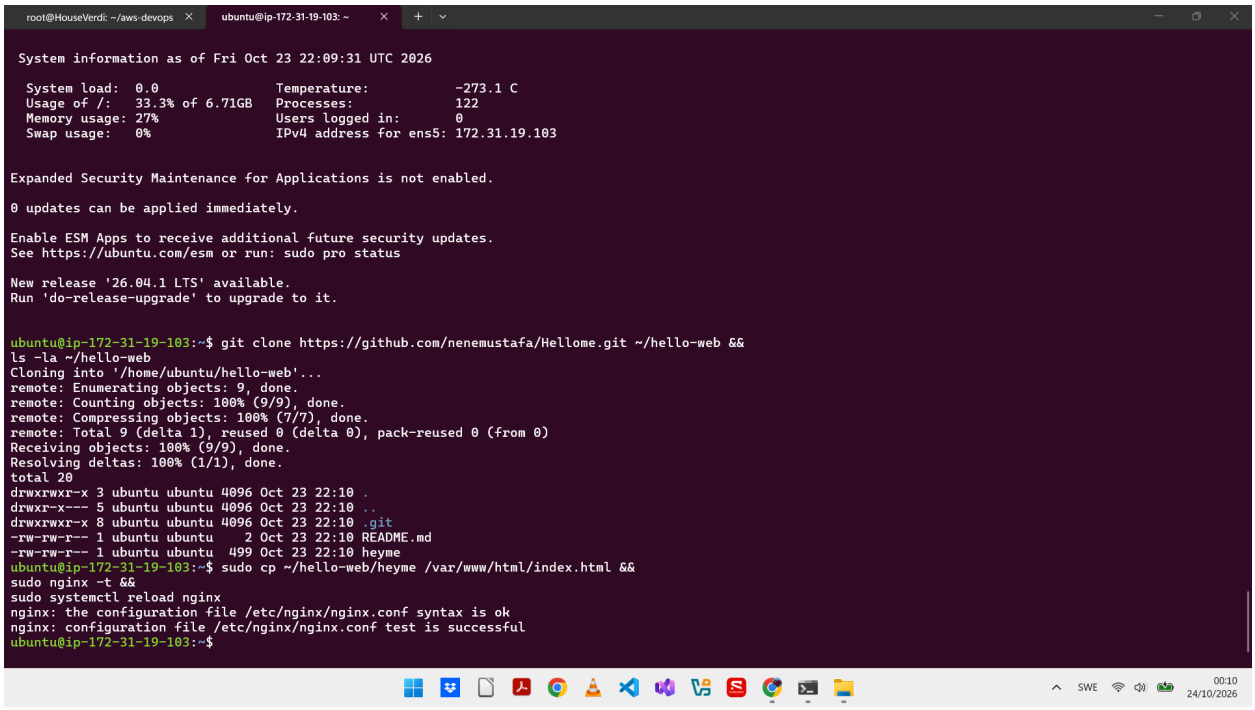


Figure 10. Deployment as index.html and successful Nginx test

Local HTTP test and browser

curl -i http://localhost/ on the server returned HTTP/1.1 200 OK, Content-Type text/html and Content-Length 499. The HTML contained the heading Hey me!. I opened the same page from my computer using the instance public IP.

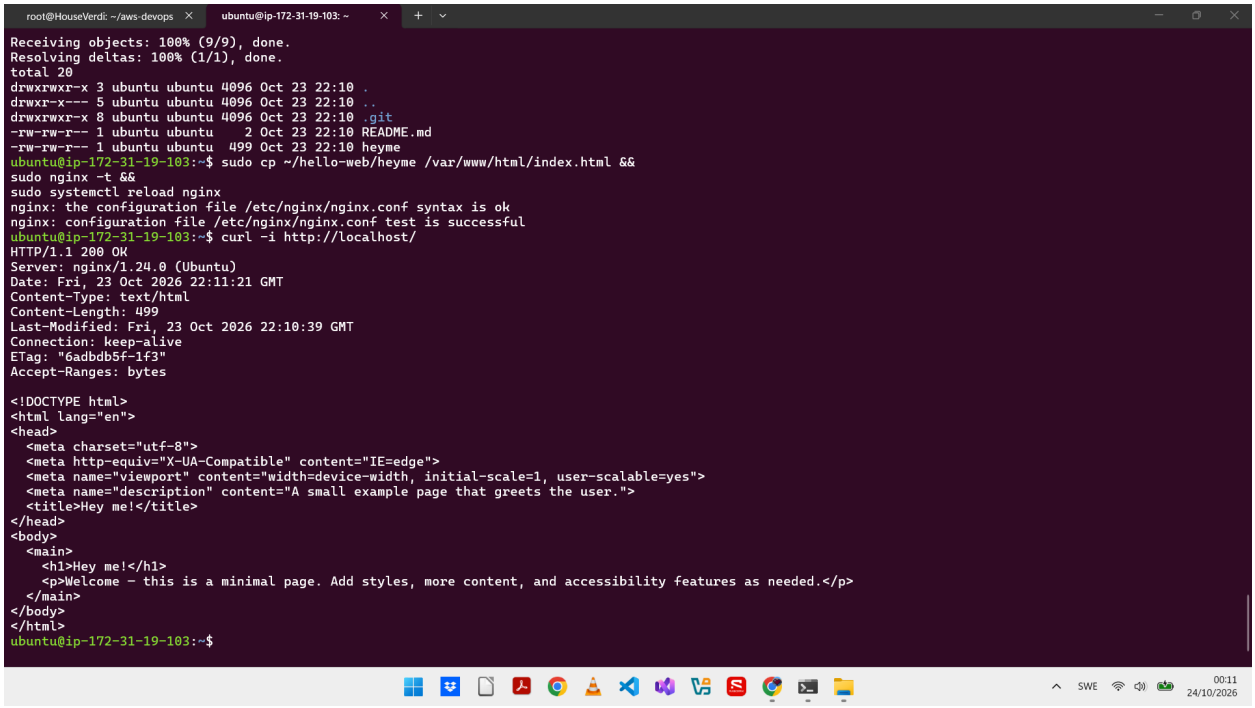


Figure 11. HTTP 200 and page HTML on the server

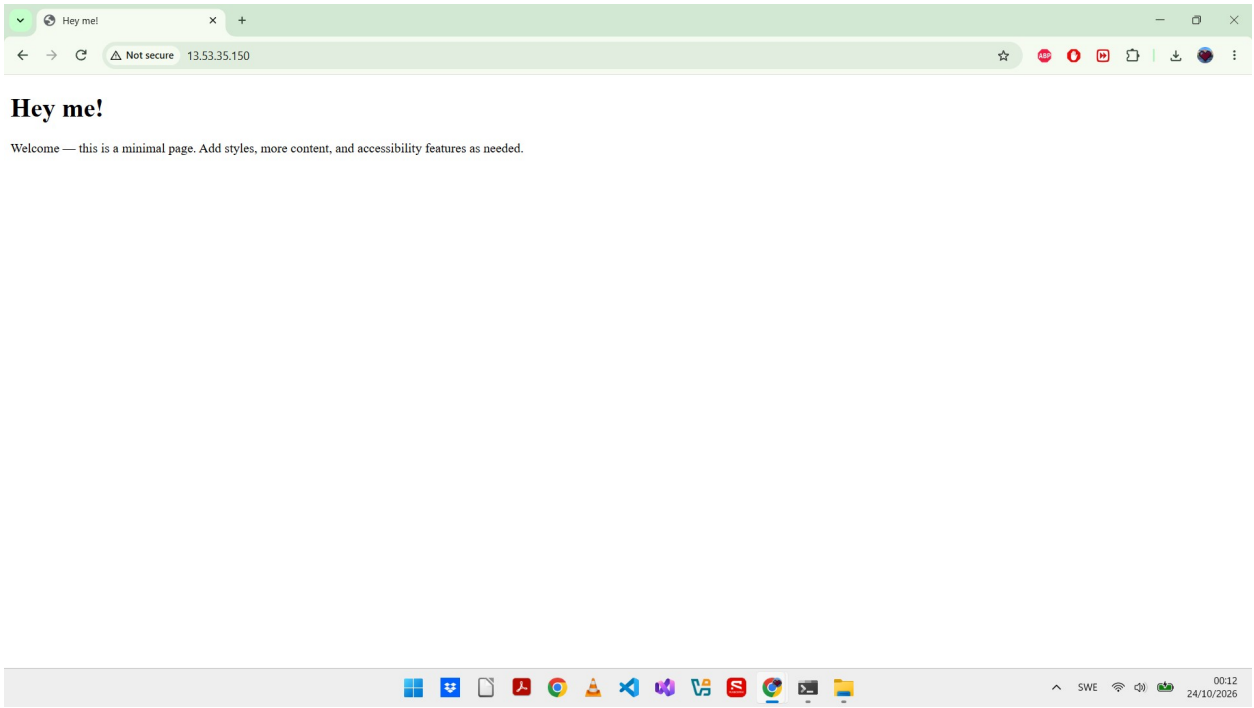


Figure 12. Hey me displayed in the browser at 13.53.35.150

External test and content check

After exiting SSH, I ran curl in my local WSL terminal against ec2-13-53-35-150.eu-north-1.compute.amazonaws.com. The response was HTTP 200 with the expected HTML. The content test returned YES and exit code 0.

I used grep -qi because the actual page text Hey me! had different capitalization from the guide's Hey Me!. I enabled pipefail so that a curl failure would also affect the test exit code.

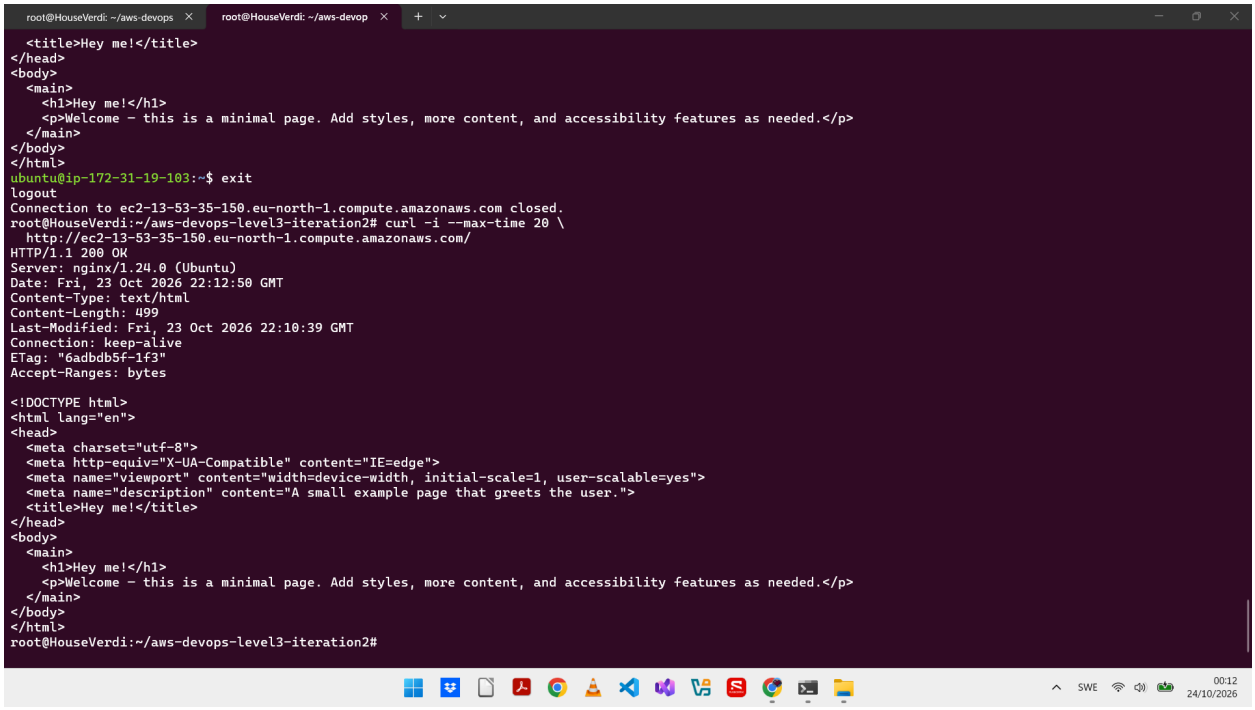


Figure 13. HTTP 200 from the local computer through the DNS name

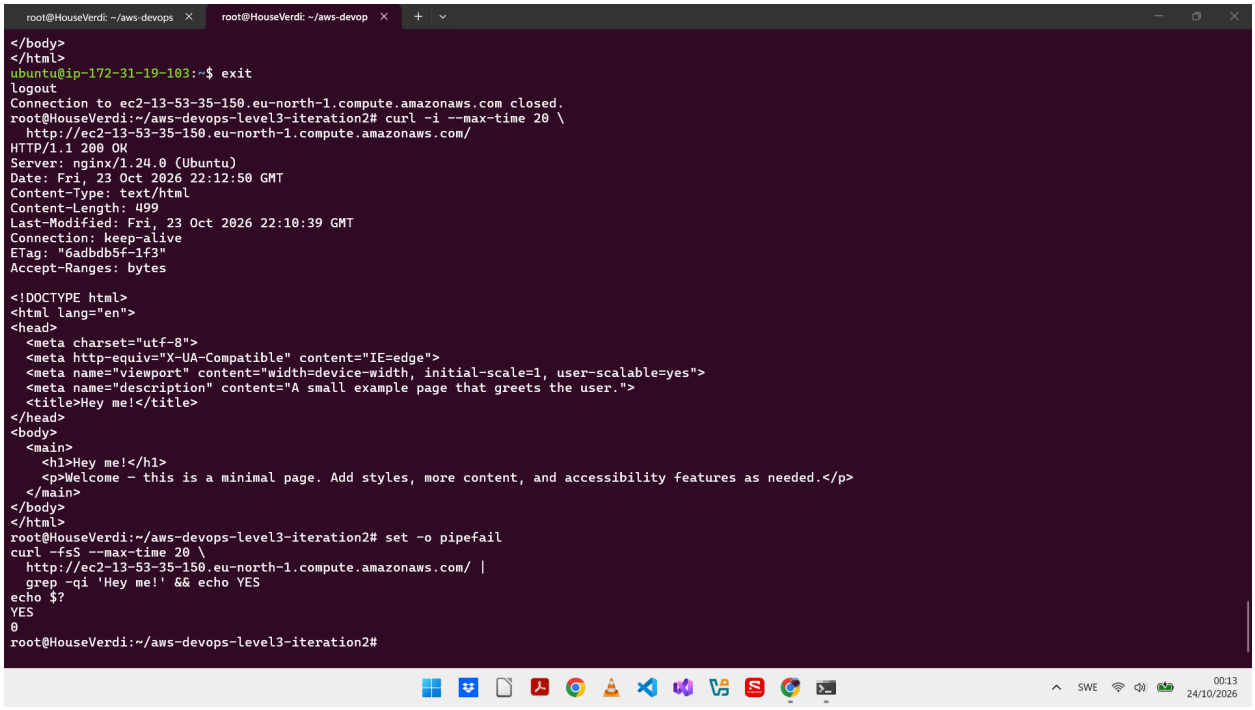


Figure 14. Automated content test with YES and exit code 0

Terraform verification and removal plan

A new terraform plan returned No changes, verifying that the infrastructure matched the configuration after deployment. This check covers the EC2 resource; I tested web content separately using HTTP.

After saving configuration, local terminal history, server history and the cloud-init log, I created a removal plan using terraform plan -destroy -out=destroyplan.

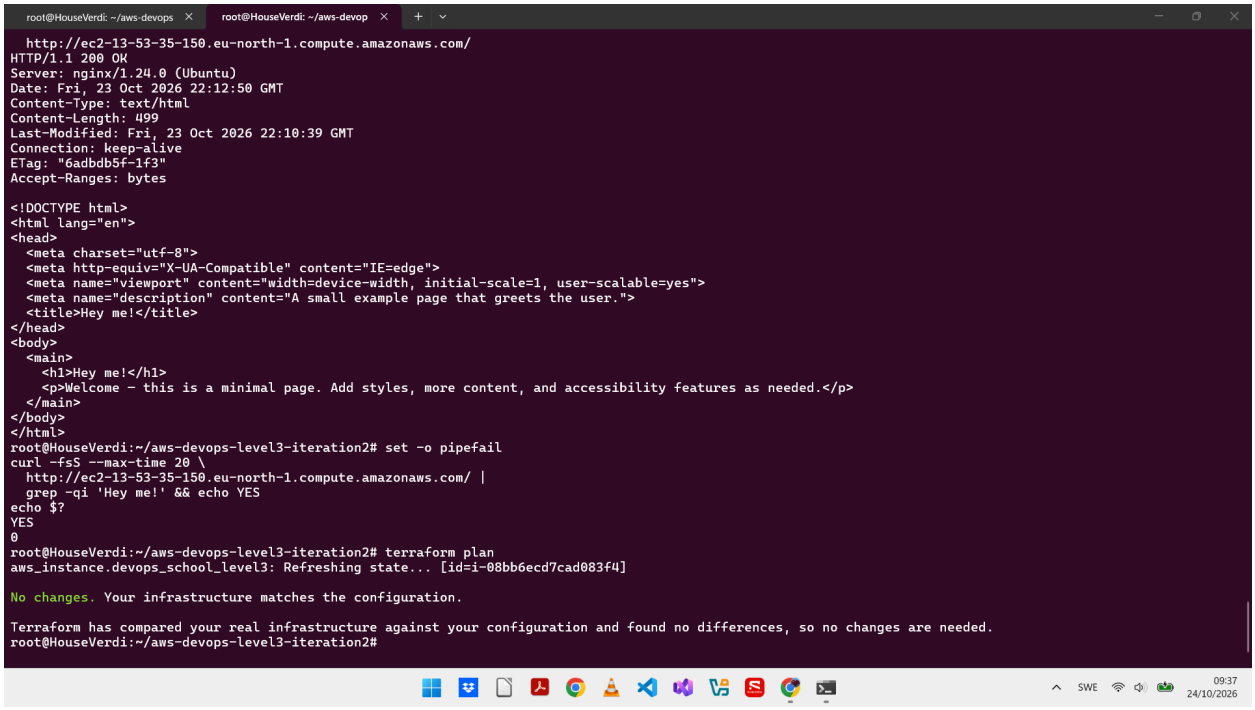


Figure 15. Terraform shows No changes

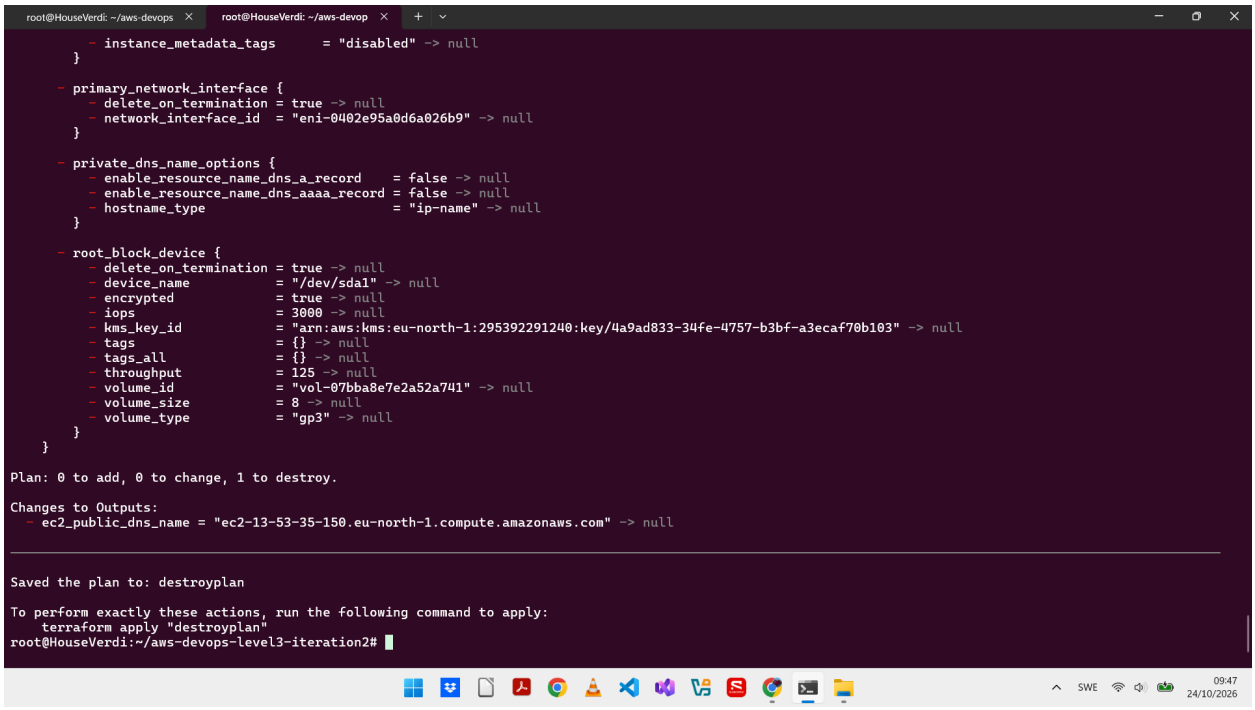


Figure 16. Removal plan with one resource and encrypted disk

Verified removal

terraform apply destroyplan removed the instance in 20 seconds and reported 0 added, 0 changed, 1 destroyed. A separate AWS CLI check returned terminated on 24 October 2026 at 09:49 Swedish time.

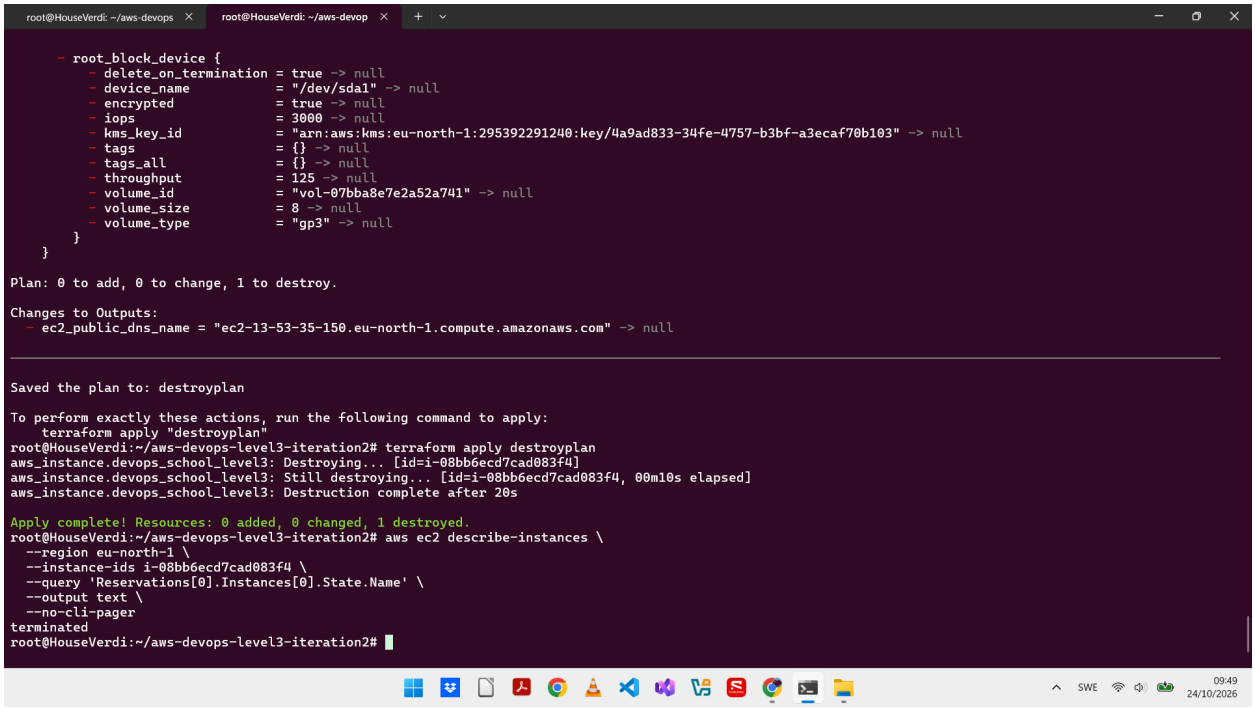


Figure 17. Successful removal and final state terminated

```
terraform plan -destroy -out=destroyplan
terraform apply destroyplan
aws ec2 describe-instances --region eu-north-1 \
  --instance-ids i-08bb6ecd7cad083f4 \
  --query 'Reservations[0].Instances[0].State.Name' \
  --output text --no-cli-pager
```

The root disk had delete_on_termination=true. The key pair and security group were outside Terraform and were not included in the removal. The IP address and DNS name in this report refer to web tests before cleanup.

Adaptations improvements and conclusion

Adaptations to the environment

The guide uses yum, service and the ec2-user account. Because the selected image was Ubuntu, I used apt-get, systemctl and the ubuntu account. The web root was /var/www/html. I adapted the file copy after inspecting the cloned contents.

Result and improvements

Iteration 2 automated package updates and web server configuration with user-data. I deployed the HTML manually, following the iteration workflow. Git was already present in the image; the installation log shows apt checking that package and installing Nginx as a new package.

Future improvements could include automatic website deployment and HTTPS. Nginx was enabled for automatic startup, but I did not perform a reboot test. The lock file from terraform init allows reuse of the same provider version.

Evidence

Reference: Level3 Iteration2 (1)(2)(1).sh. Evidence: level3-2-konfiguration.txt, level3-2-terminal-history.txt, level3-2-server-history.txt, level3-2-cloud-init.log and embedded screenshots. The local history also contains older commands; only the iteration 2 sequence is used here.

Preparation began late on 23 October. Provisioning and web tests took place after midnight on 24 October Swedish time. Server logs use UTC, so those events still show 23 October there.

Appendix Terraform files

The following files are reproduced from the exported configuration. The values in terraform.tfvars and the user-data.web.sh script belong in separate files.

providers tf

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 6.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}
```

variables tf

```
variable "aws_region" {
  type = string
}

variable "ami_id" {
  type = string
}

variable "subnet_id" {
  type = string
}

variable "main_sg_id" {
  type = string
}

variable "ec2_type" {
  type = string
}

variable "ec2_name" {
  type = string
}

variable "keypair" {
  type = string
}
```

Appendix instance and variable values

main tf

```
resource "aws_instance" "devops_school_level3" {
  ami                         = var.ami_id
  instance_type               = var.ec2_type
  subnet_id                   = var.subnet_id
  key_name                    = var.keypair
  vpc_security_group_ids      = [var.main_sg_id]
  associate_public_ip_address = true

  user_data                   = file("${path.module}/user-data.web.sh")
  user_data_replace_on_change = true

  metadata_options {
    http_tokens = "required"
  }

  root_block_device {
    encrypted = true
  }

  tags = {
    Name = var.ec2_name
  }
}
```

outputs tf

```
output "ec2_public_dns_name" {
  description = "Public DNS name of the EC2 instance"
  value       = aws_instance.devops_school_level3.public_dns
}
```

terraform tfvars

```
aws_region = "eu-north-1"
ami_id     = "ami-06fb1802e2b757f09"
subnet_id  = "subnet-05d4dca4820d2eb94"
main_sg_id = "sg-0fb87d0f08a213466"
ec2_type   = "t3.micro"
ec2_name   = "devops-school-level3.tabish-admin"
keypair    = "devops.school.level3.tabish-admin"
```

Appendix user data and server commands

user data web sh

```
#!/bin/bash
set -euxo pipefail
export DEBIAN_FRONTEND=noninteractive

apt-get update
apt-get upgrade -y
apt-get install -y nginx git

systemctl enable --now nginx
```

Manual deployment on the server

```
git clone https://github.com/nenemustafa/Hellome.git \
  ~/hello-web && \
  ls -la ~/hello-web
sudo cp ~/hello-web/heyme /var/www/html/index.html && \
  sudo nginx -t && \
  sudo systemctl reload nginx
curl -i http://localhost/
exit
```

Automatic installation checks

```
sudo cloud-init status --wait
nginx -v
git --version
systemctl is-active nginx
systemctl is-enabled nginx
sudo tail -n 60 /var/log/cloud-init-output.log
```

The checks above ran on the server through SSH from the local terminal. I saved the complete cloud-init log separately.